Caylent — Principal Cloud Architect | Whiteboarding Round Preparation

> **Company:** Bot Consulting – Caylent (Premier AWS Partner)
> **Role:** Principal Cloud Architect
> **Round:** 2nd Round — 1.5 Hour Whiteboarding Session
> **Shift:** 2-11 PM IST
> **Prepared for:** Pushparaj Naik | 22+ Years Experience

Table of Contents

---

1. Whiteboarding Round Strategy — How to Excel {#1-whiteboarding-round-strategy}

What Caylent (AWS Premier Partner) Is Looking For

Caylent is a **consulting firm** — they sell architecture expertise to enterprise clients. Your whiteboarding must demonstrate:

1. **Structured thinking** — Follow a clear framework, don't jump to AWS services
2. **Client-first language** — "The client's business requirement is..." not "I would use Lambda..."
3. **Trade-off articulation** — Show you can weigh options and justify decisions
4. **AWS depth** — As a Premier Partner, they expect service-level expertise
5. **Delivery leadership** — You're a Principal, show you can lead teams through complex deliveries

The STARS Framework for Whiteboarding

Use this for every scenario:

```
S — Scope & Requirements
    "Let me clarify the business context and non-functional requirements
first..."
    Ask: Users? Geography? SLAs? Compliance? Budget? Timeline?

T — Tradeoff Analysis
    "There are 3 approaches. Let me walk through the tradeoffs..."
    Present 2-3 options, compare, recommend one with clear reasoning

A — Architecture Design
    Draw the architecture on the whiteboard
    Start with the user/client → work inward through layers
    Label every component with the AWS service name

R — Resilience & Security
    "Let me address failure modes and security..."
    Multi-AZ, DR, encryption, IAM, networking

S — Scalability & Operations
    "Here's how this scales and how we operate it day 2..."
    Auto-scaling, monitoring, CI/CD, cost implications
```

Whiteboarding Drawing Tips

```
Always draw LEFT to RIGHT (user → system → data):

    Users → CloudFront → ALB → EKS → Aurora
                           ↓
                         SQS → Lambda → DynamoDB

Label everything:
    - Service name
    - Instance type or tier (e.g., "m6i.xlarge")
    - Key config (e.g., "Multi-AZ", "3 replicas")
    - Data flow direction (arrows)
    - Security boundary (dotted boxes)
```

---

2. Scenario 1: Enterprise Migration to AWS {#2-scenario-1-enterprise-

migration}

Prompt: "A client has a legacy on-premises application stack — 50 VMs, Oracle database, Windows/.NET services. They want to move to AWS within 6 months. Design the migration strategy."

How to Whiteboard This

Step 1: Clarify Requirements (ask out loud)

- "What's the driver? Cost reduction? Datacenter lease expiring? Scalability?"
- "What's the SLA requirement post-migration?"
- "Is there appetite for modernization or is this a lift-and-shift first?"
- "Any compliance requirements — HIPAA, PCI, SOC2?"
- "What's the team's AWS skill level?"

Step 2: Migration Strategy (draw the 7 R's)

```
┌──────────────────────────────────────────────────────────────┐
│                    AWS 7 R's Migration Strategy              │
├──────────────────┬───────────────────────────────────────────┤
│ Rehost           │ Lift-and-shift VMs to EC2 (AWS MGN)       │
│ Replatform       │ Move to managed services (RDS for Oracle) │
│ Repurchase       │ Replace with SaaS (e.g., on-prem AD → SaaS) │
│ Refactor         │ Re-architect to cloud-native (ECS/EKS)    │
│ Retire           │ Decommission unused applications          │
│ Retain           │ Keep on-prem (mainframe, regulatory)      │
│ Relocate         │ VMware Cloud on AWS (fastest)             │
└──────────────────┴───────────────────────────────────────────┘
```

Step 3: Phased Migration Architecture

```
Phase 1 (Month 1-2): FOUNDATION
├── AWS Landing Zone (Control Tower)
│   ├── Management Account (billing, org policies)
│   ├── Log Archive Account (CloudTrail, Config)
│   ├── Audit Account (SecurityHub, GuardDuty)
```

```
│   ├── Network Account (Transit Gateway, Direct Connect)
│   ├── Shared Services Account (AD Connector, CI/CD)
│   └── Workload Accounts (dev, staging, prod)
├── Networking
│   ├── Direct Connect + VPN failover
│   ├── Transit Gateway (hub-and-spoke)
│   └── Route 53 for hybrid DNS resolution
└── Identity
    └── AWS SSO + AD Connector → on-prem Active Directory

Phase 2 (Month 2-4): MIGRATE
├── Discovery: AWS Application Discovery Service / Migration Hub
├── Wave Planning: Group by dependency (database first, then app tier)
├── Migration Execution:
│   ├── VMs → EC2 via AWS MGN (Application Migration Service)
│   │   - Continuous replication (agent-based)
│   │   - Test instances → cutover window → DNS switch
│   ├── Oracle → Amazon RDS for Oracle (replatform)
│   │   OR → Aurora PostgreSQL (refactor, save licensing)
│   │   - Use AWS DMS for data migration
│   │   - Schema Conversion Tool for Oracle → PostgreSQL
│   └── File shares → Amazon FSx for Windows File Server
└── Validation: Application-level smoke tests per wave

Phase 3 (Month 4-6): OPTIMIZE
├── Right-size EC2 instances (Compute Optimizer)
├── Implement auto-scaling (ASG for EC2 fleet)
├── Enable Reserved Instances / Savings Plans
├── Set up monitoring (CloudWatch, X-Ray)
└── Security hardening (SecurityHub, GuardDuty, Config rules)
```

Step 4: Data Migration Architecture

```
On-Premises Oracle DB
    │
    ▼
AWS DMS (Database Migration Service)
    ├── Full Load + CDC (Change Data Capture)
    ├── Continuous replication during migration window
    │
    ▼
Target Options:
    Option A: Amazon RDS for Oracle (lowest risk, same engine)
        Pro: No application changes
        Con: Oracle licensing cost continues

    Option B: Aurora PostgreSQL (recommended for cost savings)
        Pro: 3x performance, no Oracle license ($$$)
        Con: Requires schema conversion + app testing
        Tool: AWS Schema Conversion Tool (SCT)

    Option C: Aurora PostgreSQL + DynamoDB (refactor)
```

```
        Pro: Best long-term architecture
        Con: Most effort, requires application refactoring
```

Key Talking Points:

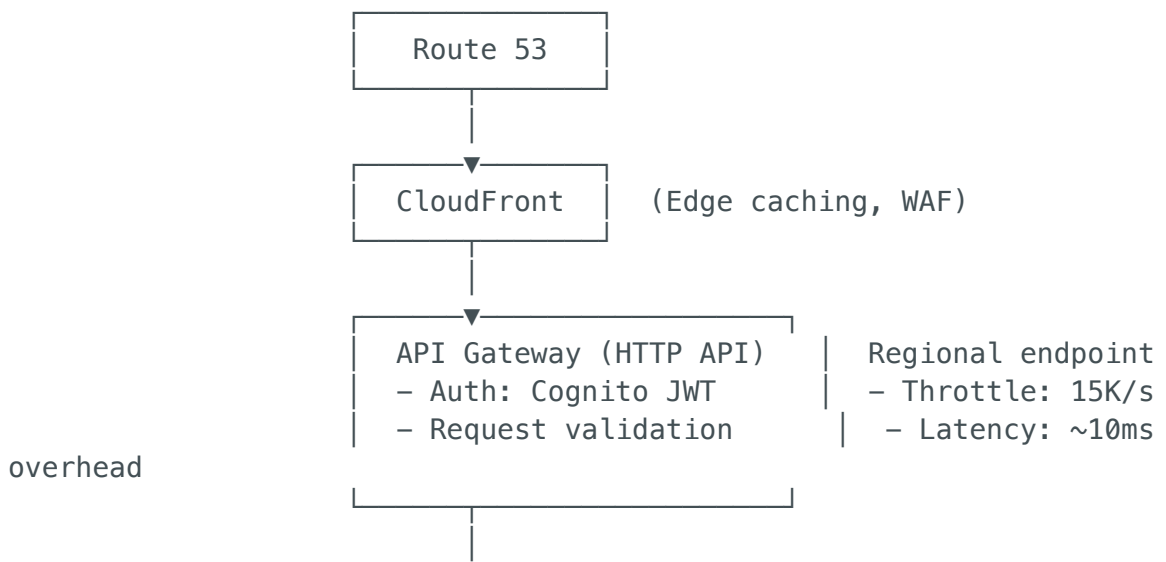
- "I would recommend a **rehost-first** strategy to de-risk the migration and meet the 6-month timeline. We can then modernize iteratively once in AWS."
- "For the Oracle database, I'd present the client with a **Build vs Buy** analysis comparing RDS Oracle licensing costs vs Aurora PostgreSQL migration effort."
- "AWS MGN gives us continuous replication with minimal downtime cutover — typically under 1 hour."

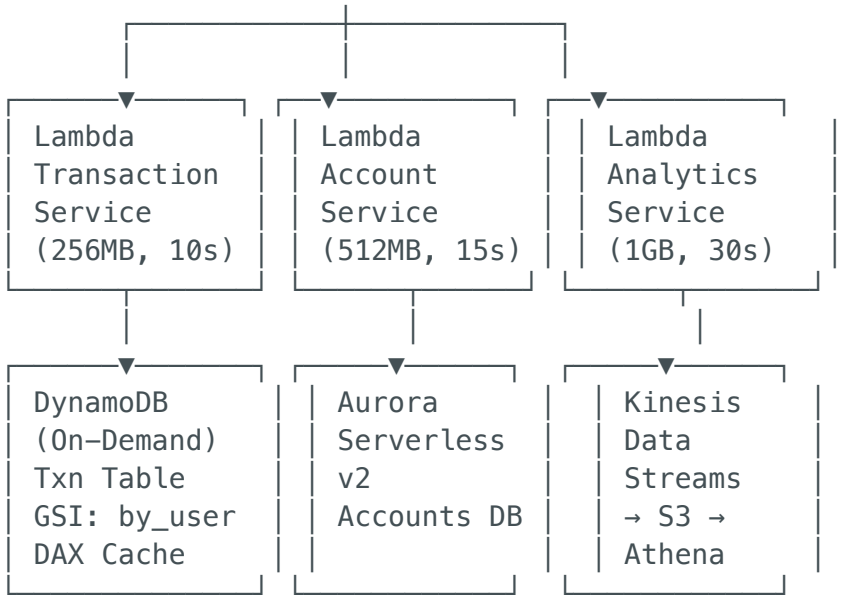
---

3. Scenario 2: Serverless-First Architecture on AWS {#3-scenario-2-serverless-first-architecture}

Prompt: "Design a serverless API backend for a fintech client that processes 10,000 transactions per minute with sub-100ms latency."

Architecture





Key Design Decisions

Decision	Choice	Why
API Gateway type	HTTP API (not REST API)	60% cheaper, lower latency, sufficient for JWT auth
Database for transactions	DynamoDB (not Aurora)	Single-digit ms latency at any scale; on-demand capacity
Caching	DAX (DynamoDB Accelerator)	Microsecond latency for hot reads; transparent to application
Cold start mitigation	Provisioned Concurrency = 50	Fintech can't tolerate cold starts; 50 warm instances for baseline
Async processing	Kinesis → Lambda → S3	Transaction analytics processed asynchronously; doesn't block the API
Authentication	Cognito + API Gateway authorizer	Managed JWT validation at the gateway; no Lambda invocation for auth

Performance Budget

```
Total latency budget: 100ms
```

```
API Gateway overhead:    ~8ms
Lambda cold start:        0ms (Provisioned Concurrency)
Lambda execution:       ~20ms
DynamoDB write:         ~10ms (single-digit ms)
DAX read:                ~1ms (microsecond)
Network overhead:        ~5ms
──────────────────────────────
Total:                  ~44ms (well within 100ms budget)
```

Scaling & Cost Estimate

```
10,000 TPS × 60 seconds = 600K requests/minute

Lambda: $0.20/1M requests + compute time
  = 600K × 60 × 24 × 30 = ~25.9B requests/month
  = ~$5,180/month (requests) + ~$3,000 (compute)

DynamoDB (On-Demand):
  = 10K WCU × 3600 × 24 × 30 × $1.25/1M WRU
  = ~$10,000/month

Total estimated: ~$18K-20K/month

Compared to EC2-based: ~$25-30K/month (including HA, ops overhead)
Savings: ~30% + zero ops overhead
```

---

4. Scenario 3: Multi-Account Landing Zone Design {#4-scenario-3-multi-account-landing-zone}

Prompt: "Design an AWS multi-account landing zone for an enterprise with 200 developers, 50 applications, and strict compliance requirements."

Architecture

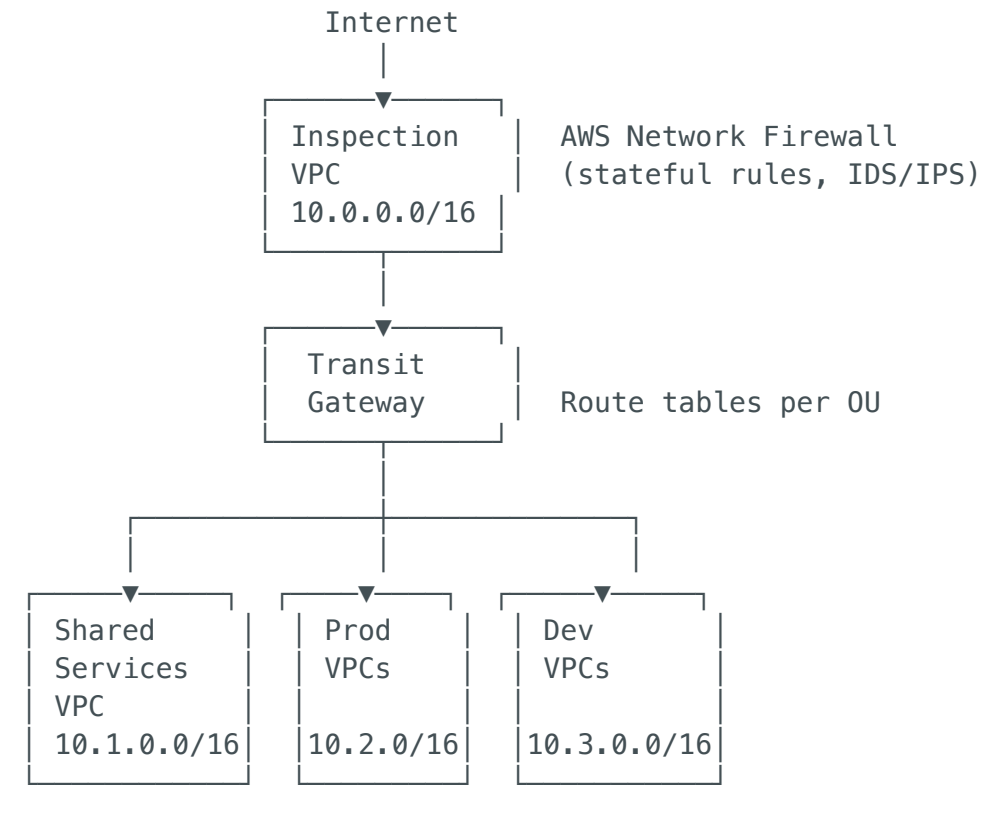
```
AWS Organization Root
├── Management Account (billing, org policies, SSO)
│
├── Security OU
│   ├── Log Archive Account
│   │   └── CloudTrail, Config, VPC Flow Logs, S3 access logs
│   ├── Security Audit Account
│   │   └── GuardDuty (delegated admin), SecurityHub, Inspector
│   └── Security Tools Account
│       └── WAF centralized, Firewall Manager
│
├── Infrastructure OU
│   ├── Network Account (Hub)
│   │   ├── Transit Gateway
│   │   ├── Direct Connect Gateway
│   │   ├── Route 53 Private Hosted Zones
│   │   ├── Network Firewall (inspection VPC)
│   │   └── VPN endpoints
│   └── Shared Services Account
│       ├── Active Directory / Identity Center
│       ├── ECR (shared container registry)
│       ├── Artifact repositories
│       └── CI/CD tooling (CodePipeline, GitHub Actions runners)
│
├── Workload OU
│   ├── Development OU
│   │   ├── App-A-Dev Account
│   │   ├── App-B-Dev Account
│   │   └── Sandbox Accounts (auto-provisioned per developer)
│   ├── Staging OU
│   │   ├── App-A-Staging Account
│   │   └── App-B-Staging Account
│   └── Production OU
│       ├── App-A-Prod Account
│       └── App-B-Prod Account
│
├── Deployment OU
│   └── CI/CD Accounts (pipeline execution, cross-account roles)
│
└── Suspended OU (quarantine for compromised accounts)
```

Service Control Policies (SCPs)

```
// SCP: Deny actions outside approved regions
{
  "Version": "2012-10-17",
  "Statement": [{
    "Sid": "DenyOutsideApprovedRegions",
    "Effect": "Deny",
    "Action": "*",
    "Resource": "*",
```

```
    "Condition": {
      "StringNotEquals": {
        "aws:RequestedRegion": ["us-east-1", "us-west-2", "eu-west-1"]
      },
      "ArnNotLike": {
        "aws:PrincipalARN": "arn:aws:iam::*:role/OrganizationAdmin"
      }
    }
  }]
}

// SCP: Prevent disabling security services
{
  "Version": "2012-10-17",
  "Statement": [{
    "Sid": "PreventSecurityDisable",
    "Effect": "Deny",
    "Action": [
      "guardduty:DeleteDetector",
      "guardduty:DisassociateFromMasterAccount",
      "config:StopConfigurationRecorder",
      "config:DeleteConfigurationRecorder",
      "cloudtrail:StopLogging",
      "cloudtrail:DeleteTrail"
    ],
    "Resource": "*"
  }]
}
```

Network Architecture (Hub-and-Spoke)



```
TGW Route Tables:
  - Production RT: routes to Shared Services + Inspection VPC only
  - Dev RT: routes to Shared Services + Internet (via NAT)
  - Prod cannot reach Dev (network isolation)
```

Account Vending Machine

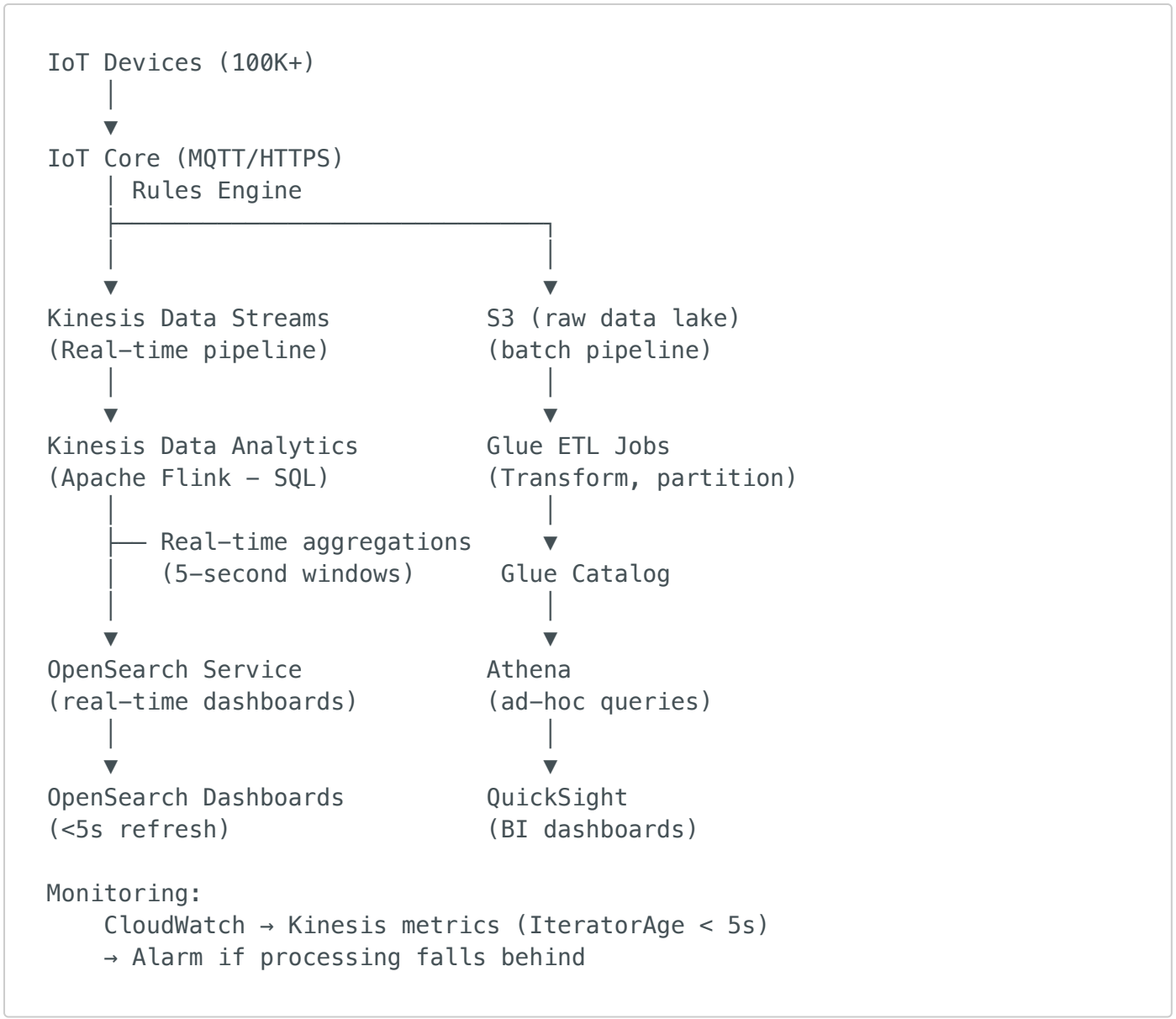
```
Developer requests new account via ServiceNow / Backstage
    |
    ▼
Step Functions workflow:
    1. Validate request (approved manager, budget code)
    2. Create AWS Account via Organizations API
    3. Move to correct OU (Dev/Staging/Prod)
    4. Apply baseline CloudFormation StackSet:
       - VPC with standard CIDR
       - TGW attachment
       - CloudTrail, Config, GuardDuty enabled
       - Default IAM roles (Admin, Developer, ReadOnly)
       - Tagging enforcement (Config rules)
    5. Create DNS delegation (Route 53)
    6. Configure SSO permission sets
    7. Notify requestor via email/Slack
    |
    ▼
Account ready in ~15 minutes (fully automated)
```

---

5. Scenario 4: Real-Time Data Platform Architecture {#5-scenario-4-real-time-data-platform}

Prompt: "A client needs a real-time data analytics platform. They have 500GB of streaming data per day from IoT devices and need dashboards with <5 second refresh."

Architecture



Lambda Architecture (Hot + Cold Path)

```
Hot Path (Real-time):
  IoT Core → Kinesis Streams → Flink SQL → OpenSearch → Dashboard
  Latency: < 5 seconds
  Use: Live monitoring, alerts, real-time KPIs

Cold Path (Batch):
  IoT Core → Kinesis Firehose → S3 (Parquet, partitioned by date/device)
  → Glue ETL (hourly) → Athena/QuickSight
  Latency: 15-60 minutes
  Use: Historical analysis, trend reports, ML training data
```

Key Design Decisions

Decision	Choice	Rationale
Ingestion	IoT Core + Kinesis Streams	IoT Core handles MQTT; Kinesis handles scale (1MB/s per shard)
Real-time processing	Kinesis Data Analytics (Flink)	Managed Apache Flink; SQL for aggregations; exactly-once semantics
Dashboard store	OpenSearch	Sub-second query latency for time-series dashboards
Data lake	S3 + Glue Catalog	Cost-effective storage; Parquet format for analytics
Shard count	500GB/day ÷ 86400 ÷ 1MB/s ≈ 6 shards (+ buffer = 10)	Right-sized for throughput

6. Scenario 5: Modernization — Monolith to Microservices on EKS {#6-scenario-5-monolith-to-microservices}

Prompt: "A client has a Java monolith serving 5M users. They want to modernize to microservices on EKS. Design the modernization strategy and target architecture."

Strangler Fig Migration Strategy

```
Phase 1: Foundation (Month 1–2)
├── Set up EKS cluster (production-grade)
├── Set up CI/CD pipeline (GitHub Actions + ArgoCD)
├── Set up shared services (observability, service mesh)
└── Deploy monolith as a container on EKS (no changes yet)
    └── This proves: container builds, deployment pipeline, monitoring
```

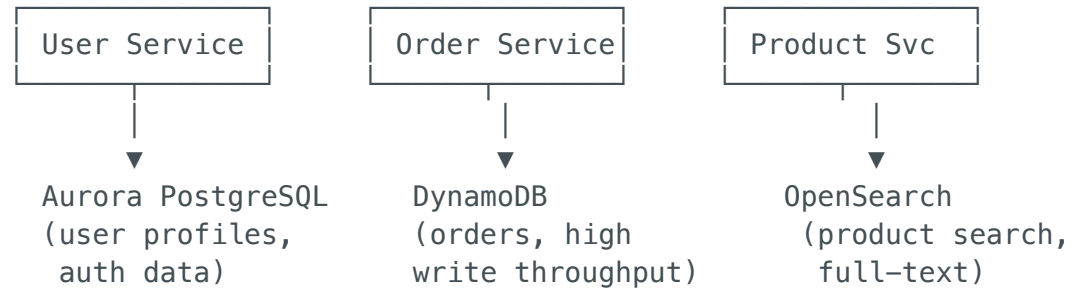
```
Phase 2: Strangler Fig (Month 3–8)
├── Identify domain boundaries (Domain-Driven Design)
│   ├── User Service (auth, profile)
│   ├── Product Service (catalog, search)
│   ├── Order Service (cart, checkout)
│   ├── Payment Service (transactions)
│   ├── Notification Service (email, SMS, push)
│   └── Analytics Service (events, reporting)
│
├── Extract one service at a time:
│   1. Start with lowest-risk, well-bounded context (e.g., Notifications)
│   2. Build as new microservice on EKS
│   3. Route traffic via API Gateway:
│      - New path → microservice
│      - Everything else → monolith
│   4. Validate with canary traffic (5% → 25% → 100%)
│   5. Remove code from monolith

Phase 3: Full Microservices (Month 9–12)
├── All services extracted
├── Monolith decommissioned
├── Event-driven communication (EventBridge + SQS)
└── Each service owns its data (database per service)
```

Target EKS Architecture

```
EKS Cluster (Private, Multi-AZ)
│
├── System Namespace
│   ├── CoreDNS, kube-proxy, VPC-CNI
│   ├── ArgoCD (GitOps controller)
│   ├── External Secrets Operator → AWS Secrets Manager
│   ├── Karpenter (node autoscaling)
│   ├── AWS Load Balancer Controller
│   ├── cert-manager (TLS)
│   └── Kyverno (policy engine)
│
├── Istio System Namespace
│   ├── istiod (control plane)
│   └── Istio Ingress Gateway
│
├── App Namespace: user-service
│   ├── Deployment (3 replicas, HPA: CPU 70%)
│   ├── Service (ClusterIP)
│   ├── VirtualService (Istio routing)
│   ├── PeerAuthentication (STRICT mTLS)
│   └── AuthorizationPolicy (allow from: order-service)
│
├── App Namespace: order-service
│   ├── Deployment (5 replicas, HPA: CPU 60%, custom: queue depth)
│   ├── SQS consumer (via KEDA ScaledObject)
```

```
│   └── ...
│
├── Observability Namespace
│   ├── Prometheus (ADOT for managed)
│   ├── Grafana
│   ├── Loki (log aggregation)
│   └── Jaeger / X-Ray (distributed tracing)
│
└── Node Groups
    ├── System: m6i.xlarge (on-demand, 3 nodes)
    ├── Application: c6i.2xlarge (Karpenter, spot + on-demand mix)
    └── Data: r6i.xlarge (on-demand, for cache-heavy workloads)
```

Database Per Service Pattern



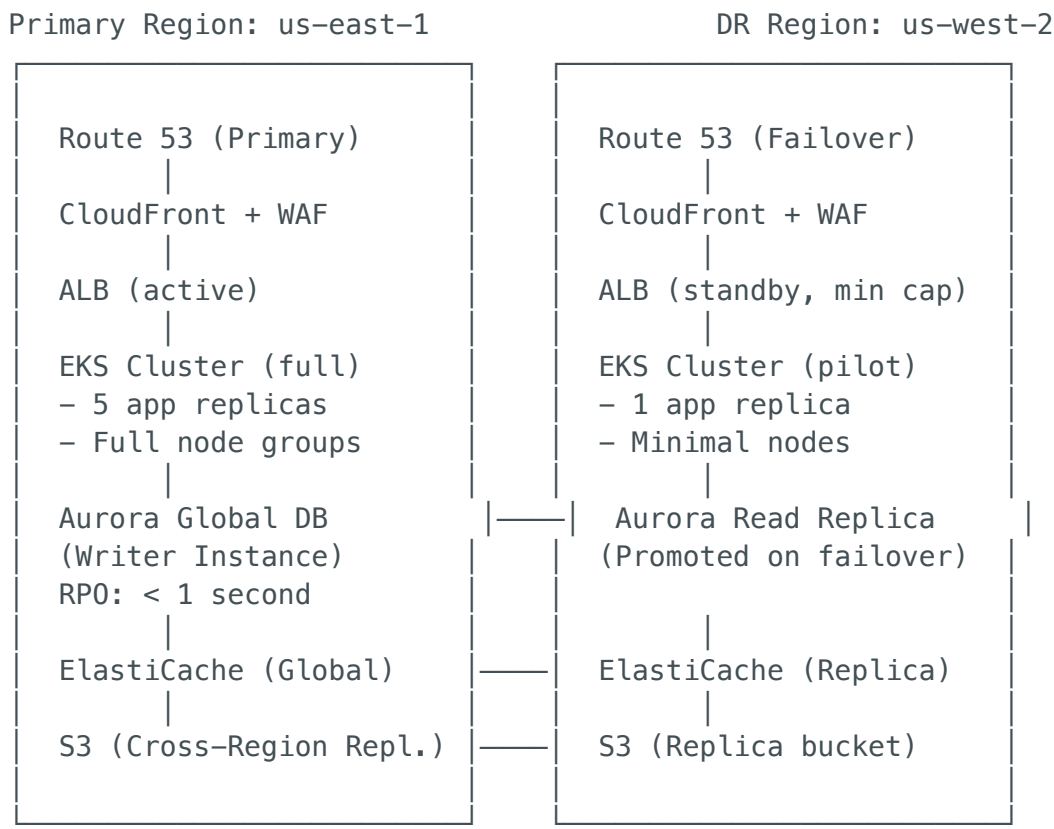

```
Communication: EventBridge (async events)
  - UserCreated → Order Service (create customer record)
  - OrderPlaced → Notification Service (send email)
  - ProductUpdated → Search Service (re-index)
```

---

7. Scenario 6: DR & High Availability Architecture {#7-scenario-6-dr-and-high-availability}

Prompt: "Design a DR strategy for a banking client with RPO < 1 minute and RTO < 15 minutes."

Multi-Region Active-Passive Architecture




```
Failover Sequence (< 15 min RTO):
1. CloudWatch alarm detects primary unhealthy (2 min)
2. Lambda triggers failover automation (1 min)
3. Aurora Global DB: promote secondary to writer (< 1 min)
4. EKS: scale up DR cluster (Karpenter: 2-3 min)
5. Route 53 health check fails → DNS failover (60s TTL)
6. ElastiCache Global Datastore: promote replica (< 1 min)
7. Validate: automated smoke tests (2 min)
Total: ~10-12 minutes
```

RTO/RPO Analysis for Banking

Component	RPO	RTO	AWS Mechanism
Database	< 1 second	< 1 minute	Aurora Global DB (async repl. lag < 1s)
File Storage	< 15 minutes	< 5 minutes	S3 Cross-Region Replication (async)
Session/Cache	< 1 second	< 1 minute	ElastiCache Global Datastore
Application	N/A (stateless)	< 5 minutes	EKS + Karpenter auto-scale

Component	RPO	RTO	AWS Mechanism
DNS	N/A	< 2 minutes	Route 53 failover (60s TTL, health check: 30s)
Secrets	< 1 minute	< 1 minute	Secrets Manager multi-region replication

DR Test Automation

```
# Quarterly DR drill (automated via Step Functions)
def execute_dr_drill():
    """
    1. Scale down primary region (simulate failure)
    2. Trigger failover automation
    3. Run full test suite against DR region
    4. Measure actual RTO/RPO
    5. Fail back to primary
    6. Generate compliance report
    """
    # Step 1: Simulate failure
    route53.update_health_check(HealthCheckId=primary_hc, Disabled=True)

    # Step 2: Wait for automated failover
    start_time = time.time()

    # Step 3: Validate DR region
    while not smoke_tests_pass(dr_endpoint):
        if time.time() - start_time > 900:  # 15 min timeout
            raise DRDrillFailed("RTO exceeded 15 minutes")
        time.sleep(10)

    actual_rto = time.time() - start_time

    # Step 4: Measure RPO (check last transaction in DR DB)
    rpo = measure_replication_lag()

    # Step 5: Generate compliance report
    generate_dr_report(actual_rto, rpo)

    # Step 6: Failback
    route53.update_health_check(HealthCheckId=primary_hc, Disabled=False)
```

---

8. Scenario 7: CI/CD & Platform Engineering {#8-scenario-7-cicd-and-

platform-engineering}

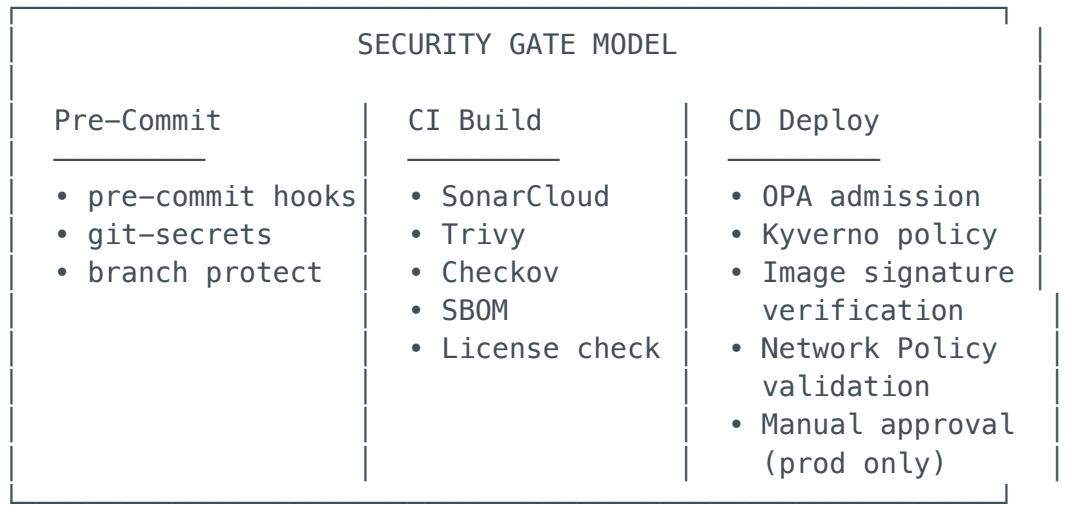
Prompt: "Design a CI/CD platform for 50 development teams deploying to EKS, with security gates and compliance controls."

Platform Architecture

```
Developer Experience Layer
├── Backstage (Internal Developer Portal)
│   ├── Service Catalog (create new service from template)
│   ├── Tech Docs (auto-generated from repo)
│   └── Scorecards (security, compliance, quality metrics)
│
├── Golden Templates (Cookiecutter / Scaffolding)
│   ├── Java Spring Boot microservice template
│   ├── Node.js Lambda template
│   ├── Terraform module template
│   └── Each includes: CI/CD, Dockerfile, Helm chart, monitoring

CI/CD Pipeline Layer
├── GitHub Actions (CI)
│   ├── Shared workflows (org-level reusable)
│   ├── Self-hosted runners on EKS (for VPC access)
│   └── Pipeline stages:
│       1. Lint + Static Analysis (SonarCloud)
│       2. Unit Tests (coverage > 80% gate)
│       3. Build Container Image (multi-stage, non-root)
│       4. Security Scans:
│          ├── Trivy (CVE scan — CRITICAL = fail)
│          ├── Checkov (IaC scan — HIGH = fail)
│          ├── TruffleHog (secrets detection)
│          └── SBOM generation (Syft)
│       5. Push to ECR (SHA-tagged, signed with cosign)
│       6. Update Helm values in GitOps repo
│
├── ArgoCD (CD — GitOps)
│   ├── ApplicationSets (auto-generate apps per team/env)
│   ├── Sync Policies:
│   │   ├── Dev: auto-sync (deploy on merge)
│   │   ├── Staging: auto-sync + auto-prune
│   │   └── Prod: manual sync (approval required)
│   ├── Progressive Delivery (Argo Rollouts):
│   │   ├── Canary: 5% → 25% → 50% → 100%
│   │   ├── Analysis: error rate < 1%, P99 < 500ms
│   │   └── Auto-rollback on failure
│   └── RBAC: team can only deploy to their namespace
```

```
Platform Services Layer
├── Crossplane (Kubernetes-native IaC)
│   └── Developers request AWS resources via K8s manifests
│       (e.g., apply RDS.yaml → Crossplane provisions RDS)
├── External Secrets Operator → Secrets Manager
├── cert-manager → ACM
├── Kyverno Policies:
│   ├── Require resource limits
│   ├── Block privileged containers
│   ├── Require approved image registries
│   └── Require labels (team, cost-center)
└── KEDA (event-driven autoscaling)
```

Security Gates in Pipeline



---

9. Scenario 8: Security Architecture & Compliance {#9-scenario-8-security-architecture}

Prompt: "Design a security architecture for a healthcare client on AWS that must comply with HIPAA."

Defense-in-Depth Architecture

```
Layer 1: Edge Security
├── CloudFront + WAF (OWASP Top 10 rules, rate limiting, geo-blocking)
├── Shield Advanced (DDoS protection with response team)
└── Route 53 DNSSEC (DNS integrity)

Layer 2: Network Security
├── VPC with private subnets only (no public-facing EC2)
├── Security Groups (stateful, reference SG IDs not CIDRs)
├── NACLs (subnet-level deny rules for additional defense)
├── VPC Flow Logs → S3 → Athena (network forensics)
├── Network Firewall (stateful inspection, IDS/IPS)
└── VPC Endpoints (S3, DynamoDB, ECR — no internet needed)

Layer 3: Identity & Access
├── IAM Identity Center (SSO + MFA enforced)
├── IAM roles with least privilege (no long-lived keys)
├── IRSA for EKS pods (pod-level AWS permissions)
├── SCPs at org level (prevent security service disabling)
└── IAM Access Analyzer (identify unused permissions)

Layer 4: Data Protection
├── Encryption at rest: KMS CMK (customer-managed keys)
│   ├── S3: SSE-KMS
│   ├── RDS/Aurora: KMS encryption
│   ├── EBS: KMS encryption
│   └── DynamoDB: KMS encryption
├── Encryption in transit: TLS 1.2+ everywhere
│   ├── ACM certificates (auto-renewal)
│   ├── Istio mTLS (pod-to-pod)
│   └── RDS: require SSL connections
├── Key rotation: automatic annual rotation (KMS)
└── Data classification: Macie (detect PHI in S3)

Layer 5: Detection & Response
├── GuardDuty (threat detection — crypto mining, credential abuse)
├── SecurityHub (unified findings dashboard, CIS benchmark)
├── Config Rules (compliance checks — 50+ rules)
├── CloudTrail (API audit logging — all regions, all accounts)
├── Inspector (EC2/ECR vulnerability scanning)
└── Detective (security investigation, entity graphs)

Layer 6: Incident Response
├── EventBridge rules for automated response:
│   ├── GuardDuty: High severity → isolate instance (Lambda removes SG)
│   ├── Config: non-compliant resource → auto-remediate
│   └── CloudTrail: root login → PagerDuty alert + SNS notification
├── AWS Systems Manager Incident Manager
└── Runbooks in SSM Automation (pre-approved response procedures)
```

HIPAA-Specific Controls

HIPAA Requirement	AWS Implementation
Access controls (§164.312(a))	IAM Identity Center, MFA, RBAC, IRSA
Audit controls (§164.312(b))	CloudTrail, VPC Flow Logs, S3 access logs
Integrity (§164.312(c))	KMS encryption, S3 Object Lock, versioning
Transmission security (§164.312(e))	TLS 1.2+, VPN/Direct Connect, mTLS
PHI protection	Macie detection, S3 bucket policies, DLP
BAA (Business Associate Agreement)	Sign AWS BAA, use only HIPAA-eligible services
Breach notification	GuardDuty → EventBridge → SNS → Security team

10. Scenario 9: Cost Optimization & FinOps {#10-scenario-9-cost-optimization}

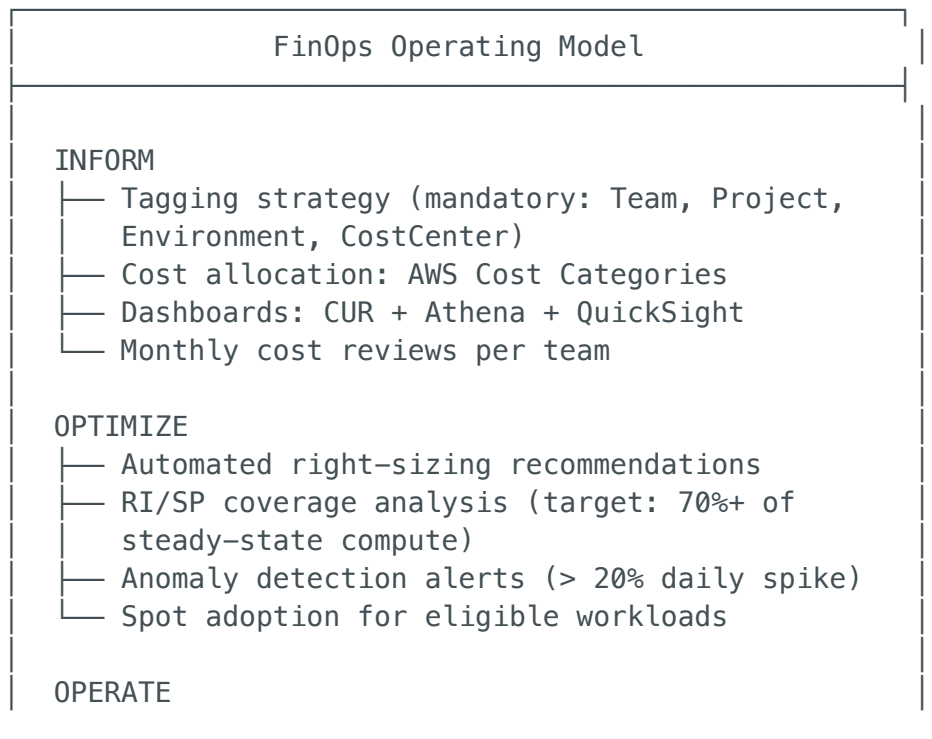
Prompt: "A client's AWS bill jumped from $200K to$500K/month. How would you investigate and optimize?"

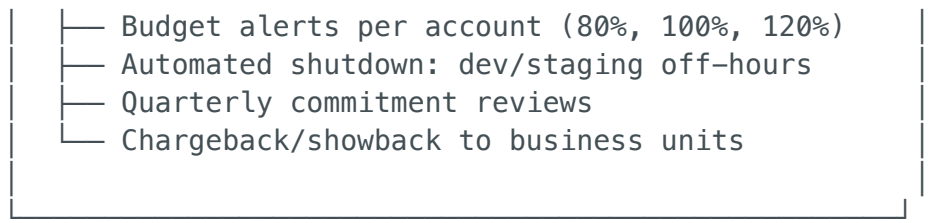
Investigation Framework

```
Step 1: Identify Cost Spike Source (15 minutes)
├── AWS Cost Explorer: filter by service, account, tag
├── Cost Anomaly Detection alerts (did it fire?)
├── Common culprits:
│   ├── EC2: Untagged instances left running
│   ├── NAT Gateway: data transfer spike
│   ├── S3: Lifecycle policy not applied, data growth
│   ├── RDS: Over-provisioned, multi-AZ in dev
│   └── Lambda: Recursive invocation, memory over-provisioned

Step 2: Right-Sizing Analysis (1-2 weeks)
├── AWS Compute Optimizer: EC2, Lambda, EBS recommendations
├── CloudWatch metrics: CPU avg < 20% = over-provisioned
```

```
├── Trusted Advisor: idle resources, low-utilization instances
└── Custom analysis:
    - RDS instances with < 10% CPU average
    - EBS volumes with 0 IOPS (unattached)
    - Elastic IPs not attached to running instances

Step 3: Implement Quick Wins (Month 1)
├── Delete unused resources: EBS snapshots, old AMIs, idle LBs
├── Right-size EC2: t3.xlarge → t3.medium where CPU < 20%
├── S3 lifecycle policies: move to IA at 30d, Glacier at 90d
├── Reserved Instances: 1-year No Upfront for stable workloads
├── Savings Plans: Compute SP for 1-year (flexibility across instance types)
├── Spot instances: Batch processing, dev/test environments
└── Schedule: Stop dev/staging instances outside business hours

Step 4: Structural Optimization (Month 2-3)
├── NAT Gateway optimization:
│   ├── VPC Endpoints for S3, DynamoDB (eliminates NAT for AWS traffic)
│   ├── Consolidate VPCs (fewer NAT Gateways)
│   └── Monitor: top talkers through NAT (VPC Flow Logs)
├── Data transfer optimization:
│   ├── CloudFront for S3 egress (cheaper than direct)
│   ├── VPC Peering same-AZ (free vs cross-AZ $0.01/GB)
│   └── Compress data in transit
├── Serverless migration for variable workloads:
│   └── EC2 cron jobs → Lambda + EventBridge
└── Container optimization:
    ├── EKS Karpenter (consolidation, right-size nodes)
    ├── Fargate Spot for non-critical workloads
    └── Graviton instances (20% cheaper, 40% better perf)
```

FinOps Operating Model



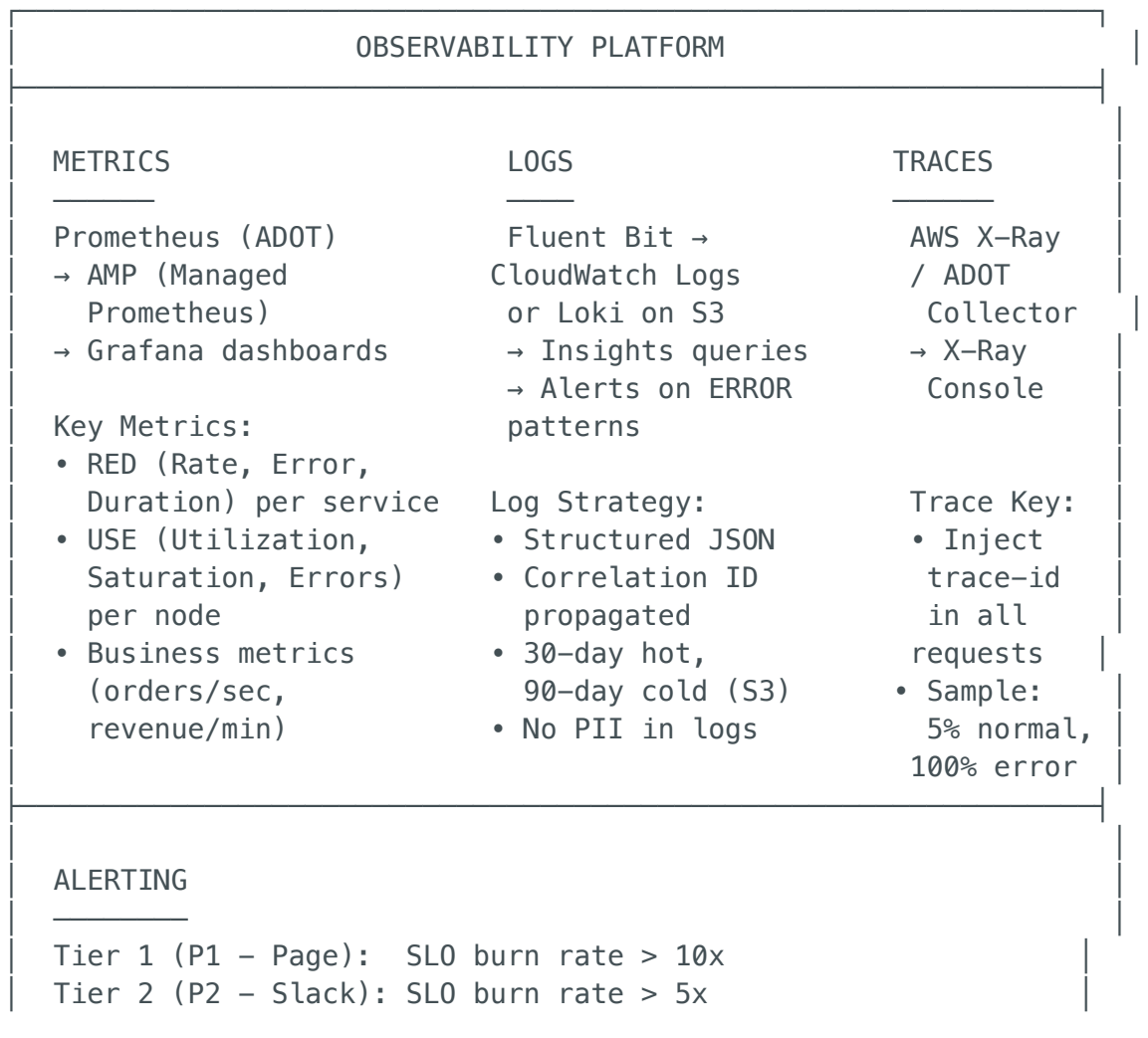


---

11. Scenario 10: Observability at Enterprise Scale {#11-scenario-10-observability}

Prompt: "Design an observability strategy for 50 microservices on EKS."

Three Pillars Architecture



```
│  Tier 3 (P3 - Ticket): Degraded performance, non-critical      │
│                                                                │
│  Alert Routing: PagerDuty → on-call engineer (15 min response) │
│  Escalation: 30 min → team lead → 60 min → VP Engineering      │
│                                                                │
└────────────────────────────────────────────────────────────────┘
```

Grafana Dashboard Hierarchy

```
Level 1: Executive Dashboard
  - System health (green/yellow/red per service)
  - SLO compliance percentage
  - Error budget remaining
  - Business KPIs (transactions/sec, revenue)

Level 2: Service Dashboard (per microservice)
  - Request rate, error rate, latency (P50/P95/P99)
  - Pod count, CPU, memory utilization
  - Upstream/downstream dependency health

Level 3: Infrastructure Dashboard
  - EKS node utilization, Karpenter scaling events
  - Aurora connections, replication lag, IOPS
  - ElastiCache hit rate, evictions, memory

Level 4: Debug Dashboard (activated during incidents)
  - Container logs (Loki queries)
  - Distributed trace waterfall (X-Ray)
  - Network flow analysis
```

---

12. Caylent-Specific & Consulting Questions {#12-caylent-specific-questions}

Q1: Why Caylent? What attracts you to a consulting role at an AWS Premier Partner?

Answer:

"Three things attract me to Caylent specifically:

1. **AWS Premier Partner status** means you're working on the most complex, enterprise-scale AWS engagements. After 22 years of building systems, I want to apply my experience across multiple industries and challenging problems — consulting gives me that variety.

2. **Cloud-native focus.** Caylent isn't doing lift-and-shift factories — you're building modern, serverless, containerized architectures. That aligns with where I see the industry going and where I add the most value.

3. **Impact multiplier.** As a Principal Architect, I can shape how multiple client organizations adopt AWS — building reusable frameworks, reference architectures, and best practices that scale across engagements. In a product company, I'd optimize one system. At Caylent, I can influence dozens."

Q2: Describe a time you led a discovery session with a client who had conflicting requirements.

Answer:

"On a recent engagement, the client's CTO wanted a multi-cloud strategy for vendor independence, while the VP of Engineering wanted to go all-in on AWS for faster delivery. The security team needed SOC2 compliance regardless.

My approach:

1. **Facilitated a structured workshop** with all three stakeholders. I drew a decision matrix on the whiteboard:

Factor	Multi-Cloud	AWS All-In
Time to market	6-9 months	3-4 months
Operational complexity	High (2 clouds)	Low (1 cloud)
Vendor lock-in risk	Low	Medium
Team skills	Need to upskill	Existing expertise
SOC2 compliance	Both support it	Simpler with 1 cloud

2. **Proposed a middle ground:** Start with AWS-only for the initial platform (meet the 4-month deadline), but design with **portability guardrails** — containerize everything (EKS), use Terraform (not CloudFormation), and keep business logic cloud-agnostic.

3. **Outcome:** Both executives agreed. We shipped the platform in 4 months on AWS. The portable architecture gave the CTO confidence that a future multi-cloud expansion was feasible without a rewrite."

Q3: How do you handle scope creep in a consulting engagement?

Answer:

"I handle scope creep through a structured change control process:

1. **SOW baseline:** Every engagement starts with a clearly defined Statement of Work — deliverables, timelines, assumptions, and exclusions.

2. **Change request process:** When a client asks for something out of scope, I acknowledge it positively ('Great idea'), then assess impact:

    - Effort estimate (hours/days)
    - Impact on timeline
    - Impact on cost

3. **Present trade-offs:** 'We can absolutely add this feature. It will take an additional 2 weeks and $X. Alternatively, we can swap it for a lower-priority item already in scope.'

4. **Document everything:** Change requests logged, approved by both parties, SOW amended.

5. **Weekly status reports:** I track scope items as 'in-scope', 'change request', or 'backlog' so the client always has visibility."

Q4: How do you estimate effort for an architecture engagement?

Answer:

"I use a **T-shirt sizing** approach for initial estimates, then refine:

Phase	Activities	Typical Duration
Discovery	Stakeholder interviews, current state assessment, requirements	2-3 weeks
Design	Architecture design, PoC, HLD/LLD documents	3-4 weeks
Build	IaC implementation, CI/CD setup, security hardening	6-12 weeks
Migrate/Deploy	Data migration, application deployment, cutover	4-8 weeks
Optimize	Performance tuning, cost optimization, handoff	2-4 weeks

Estimation technique:

1. Break into work packages (not hours)
2. Assign T-shirt sizes (S=1 week, M=2 weeks, L=4 weeks, XL=8 weeks)
3. Add 20% buffer for unknowns
4. Identify risk items that could blow up estimates (data migration, legacy integration)
5. Present as a range: 'We estimate 14-18 weeks, with the primary variable being Oracle migration complexity'"

Q5: Describe your experience with pre-sales support.

Answer:

"As a Principal Architect, I've supported pre-sales in several ways:

1. **Solution architecture for proposals:** Client RFP comes in → I design the technical solution, create architecture diagrams, write the technical approach

section, and estimate effort.

2. **Technical deep-dives with prospects:** Join sales calls to answer architecture questions, demonstrate AWS expertise, and build trust with the client's technical team.

3. **Reference architectures:** I've built reusable reference architectures for common patterns (e-commerce on EKS, data lake, serverless API) that the sales team uses in proposals.

4. **PoC execution:** When a prospect needs proof, I lead 2-week PoCs that demonstrate our capability — 'show, don't tell.'

5. **Win rate impact:** My involvement in pre-sales typically increases win rates because clients trust architects who can whiteboard their solution on the spot, which is exactly what this interview round is testing."

---

13. Stakeholder & Leadership Scenarios {#13-stakeholder-leadership-scenarios}

Q6: A client's team is resistant to adopting IaC (Terraform). How do you drive adoption?

Answer:

"Resistance usually comes from fear of change, not dislike of the technology. My approach:

1. **Understand the resistance:** Talk to the team. Are they worried about job security? Lack of skills? Fear of breaking production? Each requires a different response.

2. **Start with a quick win:** Don't start with production infrastructure. Pick something low-risk:

    - 'Let's automate your dev environment provisioning. Currently it takes 2 days. With Terraform, it'll take 15 minutes.'
    - This creates a champion within the team.

3. **Pair programming:** I sit with their engineers, write Terraform together. Show, don't lecture. After 2-3 sessions, they're writing modules independently.

4. **Prove the value with metrics:**

    - Before: 2 days to provision a new environment
    - After: 15 minutes (automated, consistent, repeatable)
    - 'We eliminated 3 configuration drift incidents last month'

5. **Create guardrails, not gates:** Provide approved Terraform modules, CI/CD pipelines that run `terraform plan` on PRs, and Checkov scans. Make it easy to do the right thing.

6. **Document and celebrate:** Write up the success story. Share it with leadership. Give credit to the team members who adopted first."

Q7: How do you mentor architects and engineers?

Answer:

"I follow a **tiered mentorship model:**

For Junior/Mid Engineers:

- Weekly 1:1s focused on technical growth
- Code review with educational comments (not just 'fix this' but 'here's why')
- Assign stretch projects with coaching guardrails
- Pair architecture sessions: 'You draw the architecture, I'll ask questions'

For Senior Engineers → Architect Track:

- Architecture decision ownership: 'You write the ADR, I'll review'
- Client-facing exposure: bring them to architecture workshops
- Encourage AWS certifications (Solutions Architect Professional, DevOps Engineer)
- Cross-project exposure: rotate across engagements

For the Team:

- Architecture Guild: bi-weekly meetup to discuss patterns, review designs

- Tech Radar: team collectively evaluates new technologies
- Brown bag sessions: team members present what they learned
- Blameless post-mortems: learning from failures, not punishing

Measuring success: I track how many engineers I've helped get promoted or certified. In my last role, 3 engineers earned their AWS SAP certification within 6 months of starting the mentorship program."

Q8: Tell me about a time you made a wrong architectural decision and how you handled it.

Answer:

"On a large-scale project, I initially chose **Apache Kafka (MSK)** for our event-driven architecture because of my familiarity with it. After 3 weeks of implementation, I realized we were over-engineering:

The problem:

- We had 12 event types, ~1000 events/second
- The team spent more time managing Kafka (partitions, consumer groups, offset management) than building features
- The managed MSK cost was $2,400/month just for the cluster

My mistake: I chose based on my past experience rather than the actual requirements.

How I handled it:

1. **Acknowledged the error openly** in a team meeting: 'I made a wrong call. Let me explain why and what I propose instead.'
2. **Proposed the fix:** Replace MSK with EventBridge + SQS — managed, serverless, pay-per-event
3. **Showed the math:** EventBridge would cost ~$100/month vs $2,400 for MSK at our volume
4. **Led the migration:** We migrated in 1 week (the event contracts didn't change, just the transport)
5. **Wrote an ADR:** Documented the decision, the reversal, and the criteria for when Kafka IS the right choice (>100K events/sec, replay requirements, complex stream

processing)

Lesson: Match the technology to the requirements, not the resume. I now always start with the simplest managed service and only escalate complexity when there's a demonstrated need."

---

14. Technical Deep-Dive Rapid Fire {#14-technical-deep-dive-rapid-fire}

These are rapid-fire questions Caylent may throw at you to test depth:

AWS Services

Q: Lambda cold start — how do you minimize it?

- Provisioned Concurrency (keep N instances warm)
- Keep deployment package small (<50MB)
- Avoid VPC placement unless necessary (adds ENI setup time)
- Use ARM/Graviton runtime (10% faster cold start)
- Avoid heavy SDKs; initialize clients outside handler

Q: API Gateway — REST API vs HTTP API?

Feature	REST API	HTTP API
Cost	$3.50/million	$1.00/million
Latency	~29ms	~10ms
Features	Full (caching, WAF, usage plans, API keys)	Limited (JWT auth, CORS, basic)
Use when	Need caching, WAF, or API keys	Simple proxy to Lambda/ALB, cost-sensitive

Q: DynamoDB — when to use single-table design?

- Single-table: When you have relational-like access patterns that need to be fast (e.g., get user + orders + reviews in one query). Uses GSIs with overloaded keys.

- Multi-table: When entities are truly independent, different teams own them, or the team doesn't have DynamoDB expertise.
- My recommendation: Start with multi-table. Move to single-table only when you've proven the access patterns through production traffic.

Q: ECS vs EKS — when to choose each?

Factor	ECS	EKS
Simplicity	Simpler, AWS-native	Complex, more features
Portability	AWS-only	Multi-cloud (K8s is portable)
Ecosystem	Limited	Rich (Helm, Istio, ArgoCD, KEDA)
Team skills	AWS ops team	Kubernetes expertise required
Cost	No cluster fee	$0.10/hr/cluster(72/month)
Choose when	Small team, AWS-only, <20 services	Large team, multi-cloud possible, need K8s ecosystem

Q: S3 storage classes — explain the tiers.

```
Standard         → Frequently accessed (default)
Standard-IA      → Access < 1x/month, 128KB minimum, 30-day minimum
One Zone-IA      → Same as IA but single AZ (non-critical data)
Glacier Instant  → Archive with millisecond access
Glacier Flexible → Archive, 1-5 minute retrieval, or bulk (5-12 hours)
Glacier Deep     → Cheapest archive, 12-48 hour retrieval
Intelligent-Tier → Auto-moves objects between tiers based on access
```

My rule: Use Intelligent-Tiering for data with unpredictable access patterns. Use lifecycle policies for data with known access patterns.

IaC & DevOps

Q: Terraform state locking — how does it work?

- State stored in S3 bucket
- DynamoDB table used for locking (partition key: LockID)
- When `terraform plan/apply` runs, it writes a lock record to DynamoDB
- If another operation is in progress, the lock prevents concurrent modifications

- Lock released on completion or timeout
- `terraform force-unlock <LOCK_ID>` for stuck locks (use carefully)

Q: How would you handle a Terraform state file that's been corrupted?

1. Don't panic. S3 versioning should be enabled on the state bucket
2. List versions: `aws s3api list-object-versions --bucket my-state-bucket`
3. Restore previous version: `aws s3api get-object --bucket my-state-bucket --key terraform.tfstate --version-id <last-good-version> terraform.tfstate`
4. Upload restored state: `terraform state push terraform.tfstate`
5. Validate: `terraform plan` (should show no changes if state matches reality)
6. Root cause: investigate how corruption happened (concurrent access? manual edit?)

Q: GitOps — explain the pull-based model vs push-based.

```
Push-based (traditional CI/CD):
  CI Pipeline → kubectl apply → Cluster
  Problem: pipeline needs cluster credentials

Pull-based (GitOps — ArgoCD):
  Developer → Git commit → ArgoCD polls Git → Syncs to cluster
  ArgoCD RUNS INSIDE the cluster
  Benefits: no external access needed, Git is source of truth,
  auto-heal if someone manually changes cluster state
```

Observability

Q: How do you detect a memory leak in a containerized service?

1. **Grafana dashboard:** Container RSS memory trending upward over time (sawtooth = GC normal; steady climb = leak)
2. **Alert:** Prometheus alert when container memory usage > 80% of limit for >30 minutes
3. **Investigate:** `kubectl top pods`, then exec into pod for profiling
4. **Root cause:** Heap dump analysis (Java: jmap; Node: --inspect; Go: pprof)
5. **Fix:** Code fix + resource limits as safety net (OOMKilled > crash-loop > cascade failure)

Q: What's the difference between CloudWatch Metrics, Logs, and Traces?

- **Metrics:** Numeric time-series data (CPU%, request count, error rate). Used for dashboards and alerting.
- **Logs:** Text/structured event records. Used for debugging, audit, forensics. Higher volume, higher cost.
- **Traces:** End-to-end request path across distributed services. Used for latency analysis and dependency mapping.
- **Relationship:** A metric alerts you (error rate spike), logs tell you what happened (stack trace), traces show you where the bottleneck is (which service in the chain was slow).

---

15. Questions to Ask the Interviewer {#15-questions-to-ask}

About the Role

1. "What does a typical Caylent engagement look like for a Principal Architect? How long are engagements typically?"
2. "How is the team structured? Do Principal Architects lead a team of consultants, or work independently across multiple clients?"
3. "What percentage of time is client-facing vs internal (pre-sales, mentoring, building frameworks)?"
4. "What's the most challenging engagement Caylent has delivered in the last year?"

About Growth & Culture

5. "How does Caylent support continuous learning? Is there budget for certifications or conference attendance?"
6. "What does the career path look like beyond Principal Architect?"
7. "How does Caylent engage with AWS — do you co-sell, do joint architecture reviews, participate in re:Invent?"

About Delivery

8. "What's the typical tech stack your teams deploy? Is there a standard Caylent reference architecture?"
9. "How do you handle knowledge transfer at the end of an engagement? What's your approach to making clients self-sufficient?"
10. "What tools does Caylent use internally for project management, communication, and documentation?"

Prepared by Antigravity AI | June 2026
Tailored for Caylent Principal Cloud Architect — Whiteboarding Round